■ Production Status & Deliverables Report

Project: Multimodal AI Assistant (M-Chatbot) **Date:** October 29, 2026 **Environment:** Local Development / Production Readiness Review

System Health: **Frontend and backend startup observed; full feature health not yet verified**

1. Executive Summary

The Multimodal AI Chatbot source archive was reviewed to prepare this status report. The project contains a React/Vite frontend and a FastAPI backend with database persistence, multimodal request handling, document ingestion and retrieval (RAG), web search, and audio-related endpoints. In the user's latest local run, Vite reported ready at **http://localhost:5173/**; Uvicorn started at **http://127.0.0.1:8000** and the application initialized its database tables using SQLite after PostgreSQL connection attempts failed.

Production status: local startup is demonstrated, but this is not equivalent to full production verification. The latest uploaded archive was statically inspected; a fresh full test-suite run, frontend production build, and live AI-provider integration test were not performed for this report. Public deployment should wait until the high-priority security and operational items below are addressed.

2	Not re-run	Not verified
LOCAL SERVICES STARTED	AUTOMATED TEST SUITE	LIVE AI INTEGRATIONS

2. Infrastructure & Runtime Matrix

Component	Technology	Endpoint / Host	Status	Operational Notes
Frontend UI	React + Vite	http://localhost:5173/	**Started**	Vite v8.3.2 reported ready in terminal. Browser-level feature verification remains pending.
Backend API	FastAPI + Uvicorn	http://127.0.0.1:8000	**Started**	Startup completed; API docs are expected at /docs if enabled by the app.
Database	PostgreSQL configured; SQLite fallback	Local SQLite file	**SQLite active**	PostgreSQL connection refused on localhost:5432 in the supplied terminal screenshot. SQLite initialized successfully for local development.
AI providers	Gemini / OpenAI paths in source	External APIs	**Not verified**	Provider credentials, model availability, quotas, and real responses must be tested in staging.
Document RAG	Extraction, chunking, retrieval	Backend API	**Code present**	Static source review indicates a RAG workflow; end-to-end retrieval quality was not tested for this report.
Web search / audio	Search + TTS / optional STT	External services	**Not verified**	Live network and provider integration checks remain outstanding.

3. Automated Test Suite Audit

Verification record for this report

The supplied archive includes test code, but a fresh pytest run was not executed while producing this report. Therefore, no pass count, test duration, or coverage percentage is claimed here. The earlier startup screenshots establish that the development servers launched; they do not prove all API routes or AI features work correctly.

```
Frontend: VITE v8.3.2 ready
Local: http://localhost:5173/

Backend: Uvicorn running on http://127.0.0.1:8000
Database: PostgreSQL connection refused on localhost:5432
Fallback: SQLite selected for local development
Application: database tables initialized; startup complete
```

Recommended test modules

Area	What to verify	Status
Health / startup	Health endpoint, database connectivity, root route	**Run required**
Chat	New conversation, multi-turn context, persistence, provider errors	**Run required**
Document RAG	Upload, extraction, chunking, retrieval relevance and source metadata	**Run required**
Safety / uploads	Invalid file types, size limits, malformed files, prompt-injection cases	**Run required**
Session isolation	Confirm one session cannot read, edit or delete another session's data	**Run required**
Web / audio	Real search responses/citations, TTS output, STT when configured	**Run required**

4. Deliverables Status

Stage 1: Implemented in source / local startup observed

✓ React/Vite frontend project and FastAPI backend entry point.

✓ Conversation and message persistence models/routes.

✓ Document upload and retrieval-oriented RAG workflow in source.

✓ Multimodal image request handling paths.

✓ Web-search and text/audio service paths.

✓ Local SQLite fallback when PostgreSQL is unavailable.

Stage 2: Validation and hardening required

■ Run the full backend test suite and save the actual pytest output.

■ Run npm run build (and lint, if configured) from a clean frontend install.

■ Verify live AI-provider credentials, model IDs, errors, timeouts and quota handling.

■ Test document upload limits, image formats, retrieval accuracy, and session boundaries.

■ Choose and configure a production database; disable silent SQLite fallback in production.

Stage 3: Production roadmap

■ Authentication and per-user authorization for all conversations and documents.

■ HTTPS deployment, restricted CORS, secrets management and dependency scanning.

■ Rate limits, structured logs, monitoring, backups and restore testing.

■ CI/CD pipeline, migrations, load testing, and documented incident recovery.

5. Production Risks & Recommendations

HIGH — Authentication / authorization

No clear account-level authentication and per-user ownership boundary was identified in the reviewed API paths. Before public multi-user hosting, enforce access checks on every conversation, document, version, and file operation.

HIGH — Database fallback

PostgreSQL refused the connection in the local run and the application switched to SQLite. Keep this behavior for development only; production should fail fast if its configured database is unavailable.

HIGH — Upload resource handling

Review upload code to ensure file size is enforced while reading, not only after the full file is in memory. Validate content type and file signatures; use generated unique storage names.

MEDIUM — Prompt injection and RAG

Heuristic prompt checks cannot guarantee safety. Treat retrieved documents and search snippets as untrusted, constrain tools, validate tool arguments, and run adversarial tests.

MEDIUM — Observability and operations

Add rate limits, request timeouts, structured logs, metrics, provider error handling, dependency scanning, backups, and recovery testing.

6. Verification Checklist

```
[✓] Frontend dev server reported ready at http://localhost:5173/
[✓] Backend Uvicorn reported running at http://127.0.0.1:8000
[✓] Database initialization completed using SQLite fallback
[ ] Frontend production build completed successfully
[ ] Full backend pytest suite executed and results recorded
[ ] Health endpoint returns expected status in browser/API client
[ ] Real text and image prompts verified with configured AI provider
[ ] Document upload and RAG retrieval tested end to end
[ ] Web search, citations, TTS and configured STT tested live
[ ] Authentication, session isolation, upload limits and production CORS verified
```

Final assessment: The project is at a local development / staging-readiness checkpoint, not yet a verified production release. The local startup evidence is positive, but full feature testing and production hardening remain necessary. This report deliberately does not claim 100% health, 100% uptime, a specific passing test count, or verified live AI integrations without supporting test evidence.

Prepared from the uploaded project archive, static source review, and the user's terminal screenshots dated October 29, 2026. This is not a penetration test or independent certification.